AI Powered Stock Market Prediction Web Application

Project Progress Report Submitted in Partial Fulfillment of the Requirements for the Degree of Bachelor of Technology in the field of Computer Science and Technology

BY
Bijoy Bhadra (123231110209)
Swarojit Mondal (123231110221)
Tania Samajpati (123231110222)

Under the supervision
of
Mrs. Saswati Rakshit
&
Dr. Manish Mukul Das

Department of Computer Science and Technology
JIS College of Engineering

Block-A, Phase-III, Kalyani, Nadia, Pin-741235
West Bengal, India

DEC, 2025

JIS College of Engineering
Block 'A', Phase-III, Kalyani, Nadia, 741235
Phone: +91 33 2582 2137, Telefax: +91 33 2582 2138
Website: www.jiscollege.ac.in, Email: info@jiscollege.ac.in

CERTIFICATE

This is to certify that **Bijoy Bhadra (123231110209), Swarojit Mandol (123231110221), Tania Samajpati (123231110222)** have submitted the progress report of their project entitled **AI Powered Stock Market Prediction Web Application,** under the guidance of **Mrs. Saswati Rakshit & Dr. Manish Mukul Das** in partial fulfillment of the requirements for the award of the **Bachelor of Technology in Computer Science and Technology** from JIS college of Engineering (An Autonomous Institute) is an authentic record of their own work carried out during the academic year 2025-26 and to the best of our knowledge, this work has not been submitted elsewhere as part of the process of obtaining a degree, diploma, fellowship or any other similar title.

---------- ---------- ----------

Signature of the Supervisor **Signature of the HOD** **Signature of the Principal**

Place:

Date:

ACKNOWLEDGEMENT

The analysis of the project work wishes to express our gratitude to **Mrs. Saswati Rakshit & Dr. Manish Mukul Das** for allowing the degree attitude and providing effective guidance in development of this project work. Her conscription of the topic and all the helpful hints, she provided, contributed greatly to successful development of this work, without being pedagogic and overbearing influence.

We also express our sincere gratitude to **Dr. Sitanath Biswas**, Head of the Department of Computer Science and Technology of JIS College of Engineering and all the respected faculty members of Department of CST for giving the scope of successfully carrying out the project work.

Finally, we take this opportunity to thank to Prof. **(Dr.) Partha Sarkar**, Principal of JIS College of Engineering for giving us the scope of carrying out the project work.

Date:

.....

Bijoy Bhadra
B.TECH in Computer Science and Technology
4thYEAR/7th SEMESTER
Univ Roll--123231110209

.....

Swarojit Mondal
B.TECH in Computer Science and Technology
4thYEAR/7th SEMESTER
Univ Roll--123231110221

.....

Tania Samajpati
B.TECH in Computer Science and Technology
4thYEAR/7th SEMESTER
Univ Roll--123231110222

List of Figures

List of Tables

CONTENTS

ABSTRACT

Financial time-series forecasting is a prominent challenge in computational finance subject to the noisy, volatile and non-stationary patterns present in the global markets until October 2023. Capturing complex multivariate relationships as well as long-range temporal dependencies is difficult for traditional econometric methods and the previously established deep learning models either require computationally-intensive sequential hierarchical training of multiple prediction models or have to sacrifice predictive power. This work presents a hybrid forecasting framework that combines the iTransformer architecture with the TimeMachine state-space mechanism provided by the Mamba model. While the iTransformer uses an inverted Transformer structure to model multivariate correlations whilst processing noise due unaligned timestamps, the TimeMachine framework is able to capture longest-term market dynamics using efficient discretized state-space equations with linear scalability.

As such, the Canonical Genetic Algorithm (CGA) must be studied to obtain a systematic hyperparameter tuning of this hybrid architecture, due to classical gradient-based optimization not being well suited for a non-convex environment. The performance of the proposed framework is validated on various high volatility financial datasets from a number of domains, including equities, commodities and foreign exchange markets, under the metrics of Mean Squared Error (MSE), Coefficient of Determination (R^2) and Accuracy of Directional Agreement (ADA). Furthermore, Explainable Artificial Intelligence (XAI) techniques are introduced to detect dominant temporal and market-specific features allowing for improved explainability and reliability in real-world scenarios of financial forecasting applications.

Keywords: Financial Time Series Forecasting, iTransformer, State-Space Models, Mamba Architecture, TimeMachine, Canonical Genetic Algorithm, Multivariate Analysis, Deep Learning.

1. INTRODUCTION

The correct predictions of stock market, foreign exchange rates and commodity prices provide a valuable supply of information services and a competitive edge to institutional investors, algorithmic trading systems and macroeconomic policymakers[1]. Nonetheless, financial markets are complex and dynamic systems characterized by a high degree of noise, behavioral irrationality and rapid, nonlinear responses to exogenous macroeconomic shocks. In the past, quantitative finance was largely driven by statistical and econometric methods like ARIMA and GARCH[2]. Despite being mathematically sound, extremely interpretable, and tractable in the low-frequency domain,

these linear models are absolutely dependent on the stationarity assumption and lack any sort of structural capability to 'learn', translate or decode any non-linear multi-scale temporal dependencies encased within modern high-frequency trading datasets[3]. The Efficient Market Hypothesis (EMH) hypothesis states asset prices fully reflect all accessible information, however, numerous empirical forecasting attempts have shown that constructed inefficiencies remain existent in markets, thus making them partially predictable.

Deep neural network (DNN) techniques, such as Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs), came along in the age of deep learning, which allowed more sophisticated recurrent gating to be used to map sequential data[4]. These architectures were able to remember internal state vectors over time steps and effectively learned short-term price momentum, creating initial baselines for deep financial forecasting. But the sequential processing nature of RNNs leads to a vanishing gradient bottleneck when dealing with longer lookback windows, making them computationally expensive and under-optimal for macroeconomic detection in cyclical multi-quarter behaviors[5]. The Transformer architecture, initially designed for NLP, soon became the most parallelizable solution. Transformers employs self-attention mechanisms which are able to compute pairwise dependencies between temporal tokens simultaneously.

However, the theoretical superiority of Transformers does not match with their wider empirical applicability, where recent literature has uncovered a stunning "Transformer paradox" in multivariate time series forecasting: highly parameterized standard Transformers are very often outperformed by comparatively simple linear models[5]. This decay occurs as standard Transformers operate cross-sectionally on variates at a single timestamp, conglomerating independent physical measurements and misaligned market events into a homogeneous vector that obliterates variate-level representations. To address this innate structural deficiency, here we explore two contemporary paradigms[6]. The first is called iTransformer, it performs a radical architectural inversion by embedding time series of each variate separately into their own independent-style tokens all while utilizing the attention mechanism for cross cross-variate correlations, instead of across time steps[7]. Second, the incorporation of State-Space Models (SSMs), and in particular as here it is based on a two-tier TimeMachine architecture requiring only first-order diff equations discretized for deep learning, with linear inference complexity together with very strong long-term prediction capabilities. In addition, the hybrid networks are more complicated1 and will require advanced parameter tuning, which inspires using new methods such as Canonical Genetic Algorithms (CGAs)[9].

This study aims to solve modern financial time-series forecasting theoretical, architectural, and empirical problems. This study presents a systematic analysis on the mathematical foundations and

architectural innovations of iTransformer- and TimeMachine-type nonlinear state-space estimation models to understand how they mitigate the drawbacks associated with traditional recurrent neural network (RNN) and Transformer architectures in processing high-frequency financial data series[9][10]. A Canonical Genetic Algorithm (CGA) is used to reduce human effort in hyperparameter optimization in a given deep learning architecture. The framework incorporates a combination of financial data of varied modalities, such as market indicators and sentiment analysis, in assessing forecasting performance using a set of error metrics, uncertainty estimation and directional prediction accuracy on various financial datasets[11].

The report is structured in a systematic manner to get into theoretical and experimental aspects of the proposed forecasting framework. The Literature Review reviews the evolution of financial forecasting e.g. recurrent neural networks, Transformer and state-space models. Experimental Framework: Describes hardware-software at scale, dataset integration, optimization strategy. The Proposed Methods, including a mathematical description of iTransformer, TimeMachine and Canonical Genetic Algorithm. Methodology: Calls out data collection, preprocessing and feature engineering approaches, model training and evaluation. The Results and Discussion section assess forecasting performance across various datasets, whereas the Conclusion synthesizes key findings and contributions. In this section you are emphasizing on your research limits and how it can be further enhanced (Last but not the least, the references section).

2. LITERATURE SURVEY

The past decade, a number of paradigm-shifting transformations have occurred within the trajectory of sequence modeling in quantitative finance. This shift from linear stationarity-dependent models to non-linear recurrent networks, and now attention-based and state-space continuous architectures is indicative of a perpetual modelling effort: moving towards capturing both transient micro-structural noise as well as macro-economic drift[12]. We organize various strands of recent foundational research into five independent such topical areas in this literature review: weaknesses of recurrent networks, the development of new generations Transformer architecture, revival of state-space modeling across nearly all modalities with extensive cross-field results through emergent transfer learning benchmarks based on large language datasets (LLD), computational/algorithmic choices in hyperparameter tuning via evolutionary algorithms[13].

The foundation of deep learning in finance, recurrent neural networks (RNNs) and their more sophisticated relatives, LSTMs and GRUs established the earliest deep learning benchmarks for financial forecasting. Research by Xiao et al. (2024) report that LSTMs get very high directional

stock price prediction accuracy, generally above 90% under tightly constrained short-horizon environments. Moreover, Mun et al. Studies of MSE and MAE on datasets with more high volatility (such as Tesla equities) (2025) between LSTMs and early Transformers found that LSTMs still overall win. In particular, they found a MSE of 1.23 for the LSTM and MAE of 0.98 which illustrates how quickly (short term) reflective the architecture can be to local variations. However, the same study showed that Transformers had a greater Coefficient of Determination (R2 = 0.87) and Accuracy of Directional Agreement (ADA = 78%), indicating their strength at long-range dependency modelling and directional agreement[14]. However, RNN's sequential processing design has causes a vanishing gradient bottleneck for long lookback windows. Therefore Recurrent Neural Networks are not optimal for capturing cyclical, multi-quarter macroeconomic signals.

Combining the self-attention mechanism with the Transformer architecture appeared to resolve most of these limits on sequence length, allowing all input elements to be analysed in parallel. Yet, standard Transformers for MTSF turned out to have some structural drawbacks[15]. As elucidated by Liu et al. (2023), common implementations represent several financial metrics from a single point in time as one temporal token. This single embedding combines well-separated market events and different physical measurements resulting in pointless attention maps and poor performance for long lookback windows. This structural defect was the exact motivation behind the design of iTransformer. The iTransformer tokenizes each specific variate as a separate token by inverting the tokenization axis, embedding the entire history of that variate[16]. The attention mechanism is then used in the variate dimension to explicitly model multivariate correlations, while feed-forward networks (FFNs) handle temporal dynamics. This permutation equivariant architectural variation consistently outperforms the standard Transformers on complex real-world datasets, particularly high-dimensional traffic and financial benchmarks.

At the same time, State-Space Models (SSMs) have returned from classical control theory, where SSMs were suited to systems like the Apollo navigational systems, and then re-engineered into deep learning. The Mamba architecture is an example of a modern structured state-space model that conveys economical flexibility in selective retention of relevant financial signals over time horizons whilst isolating stochastic noise with linear-time complexity[17]. Ahamed and Cheng (2024) recently proposed the TimeMachine framework to implement the quadruple-Mamba architecture of this paradigm for long-term multivariate forecasting. This design explicitly accommodates channel-mixing (i.e. the interacting variables driving the forecast) and channel-independence (where a strong asset momentum dominates). Additional developments by Pessoa et al. By way of two-component networks that predict point estimates and the predictive variance, you reduce Kullback–Leibler divergence at scale to address the uncertain probability of Mamba models.

Recent improvements in broadcasting accuracy have also been pushed by integrated multi-modal data. Numerical data is only part of the story because financial markets are essentially driven by events. Research by Al Ridhawi et al. compared with context-based accumulation model, the empirical paper (2026) show that textual info fusion through Bidirectional Encoder Representations from Transformers (BERT) embedding and history price data compression through node transformer architectures outperforms dramatically in terms of prediction error. Their integrated model reached a mean absolute percentage error (MAPE) of 0.80% for one-day-ahead S&P 500 predictions, ARIMA and LSTM had MAPE = 1.20% and MAPE = 1.00%, respectively. Likewise, the multimodal iTransformer models when applied to Forex and Gold datasets obtained reductions in MSE of 26.79% when dynamically aligning the textual embeddings from economic calendars with temporal price information[18].

Thrid, the complexity of these hybrid networks requires careful parameter tuning. Traditional methods such as grid search become infeasible in high-dimensional spaces due to the curse of dimensionality. Thus, Canonical Genetic Algorithms (CGAs), due to the idea of natural selection also are heavily used through rallying over the crossover and mutation operations. Shreyas S., (2026**)** showed that hyperparameter optimization of deep sequence models using GAs on the Nifty 50 index improves predictive accuracy greatly, giving a reduction of over 50% RMSE as compared to manual tuning. Other studies on smart grid fraud detection using GA and XGBoost demonstrated significant performance gains in terms of accuracy, precision, and AUROC metrics[11].

A summary of existing methods for financial time-series forecasting is provided in Table 1. It summarizes architectures, datasets, empirical results and limitations on models like the iTransformer, TimeMachine (Mamba SSM), probabilistic state-space frameworks and GA-based optimization methods. The papers reviewed illustrate enhancements in forecasting performance and temporal representation yet revealed challenges with the computational overhead, scalability, and optimization convergence.

Table 1: Comparative Literature Survey on Stock Market Prediction

Author & Year	Architecture / Focus Area	Dataset & Domain Used	Key Contributions & Empirical Findings	Limitations & Constraints
Liu et al., 2023	iTransformer	Multivariate Time Series Benchmarks	Proposed axis inversion. Embedded individual series as variate tokens. Solved the temporal tokenization bottleneck.	Model efficacy degrades on highly sparse datasets without advanced

			Improved lookback scaling and achieved state-of-the-art across 6 datasets.	imputation mechanisms.
Ahamed & Cheng, 2024	TimeMachine (Mamba SSM)	Long-Term MTSF	Leveraged Mamba SSMs for long-term forecasting. Introduced a 4-Mamba architecture to handle both channel independence and mixing with linear scalability ***O(T)***.	Implementation requires precise mathematical discretization of continuous state-space matrices.
Mun et al., 2025	LSTM vs. Transformer Comparative Study	Tesla Daily Stock Data (2020-2022)	Evaluated short vs long-term efficacy. LSTM achieved lower MSE (1.23) & MAE (0.98). Transformer achieved higher R^2 (0.87) & ADA (78%).	Evaluated isolated models; did not utilize hybrid optimization or multi-asset macro factors.
Al Ridhawi et al., 2026	Node Transformer + BERT Sentiment	20 S&P 500 Stocks (1982-2025)	Extracted sentiment from social media/reports. Achieved MAPE of 0.80% for one-day-ahead predictions. Directional accuracy reached 65%.	High computational overhead due to the dual processing of large language models and node graphs.
Shreyas S., 2026	Genetic Algorithm Optimization	Nifty 50 Index (2013-2022)	Applied GA to optimize hyperparameters for deep learning models. Reduced RMSE by >50% on sequential models vs manual tuning.	CNN-LSTM hybrid models showed slight performance drops, indicating specific structural GA constraints.
Pessoa et al., 2024	Mamba-ProbTSF (Probabilistic SSM)	Synthetic & Real-world Benchmarks	Introduced a dual-network framework to quantify predictive uncertainty in Mamba. Reduced	Gaussian assumptions of residuals may not hold during

			Kullback-Leibler divergence to $\mathbf{10^{-3}}$.	extreme "black swan" financial events.
Zou et al., 2026	FT-iTransformer	6 Stock Datasets	Integrated Time-Frequency domain collaborative analysis with iTransformer using Multi-scale Temporal Convolution Networks (TCN) prior to tokenization.	High computational complexity introduced during the initial Short-Time Fourier Transform phase.

3. EXPERIMENTAL FRAMEWORK

Building an enterprise-grade forecasting engine which aligned with the stochastic nature of global financial markets requires a heavily engineered experimentation pipeline. It has to be able to ingest diverse data streams in parallel, perform wisdom-of-the-crowds type advanced topological embeddings without running into memory bottlenecks, and dynamically tune millions of parameters while avoiding both data leakage and look-ahead bias[19].

3.1 System Architecture and Design Overview

The proposed hierarchical architecture works as a sophisticated multi-stage forecasting pipeline that splits into two parallel deep-learning components, iTransformer and TimeMachine state-space framework[6]. The first layer is a data orchestration layer that standardizes numerical financial data and semantic textual vectors without temporal implications, by processing historical market information through various temporal feature extraction mechanisms[20]. After rigorous normalization and preprocessing, it is sliced into overlapping sequential windows in order to learn through time.

The inverted attention layers of the iTransformer, which account for highly embedded cross-sectional asset correlations, simultaneously process these synchronized sequences with one another along with the discretized state-space layers of the TimeMachine model that detects long-term temporal dependencies and momentum patterns[21]. An adaptive fusion layer integrates latent representations from both architectures and utilizes rolling validation performance to dynamically modulate the gradient contributions of each architecture. Atop the entire pipeline is our Canonical Genetic Algorithm (CGA) optimization engine which runs iteratively to conduct retraining cycles,

testing a variety of hyperparameter chromosomes (e.g., learning rates; network depth; dropout configurations) to achieve optimal architecture for deployment[8], as demonstrated in Figure 1.

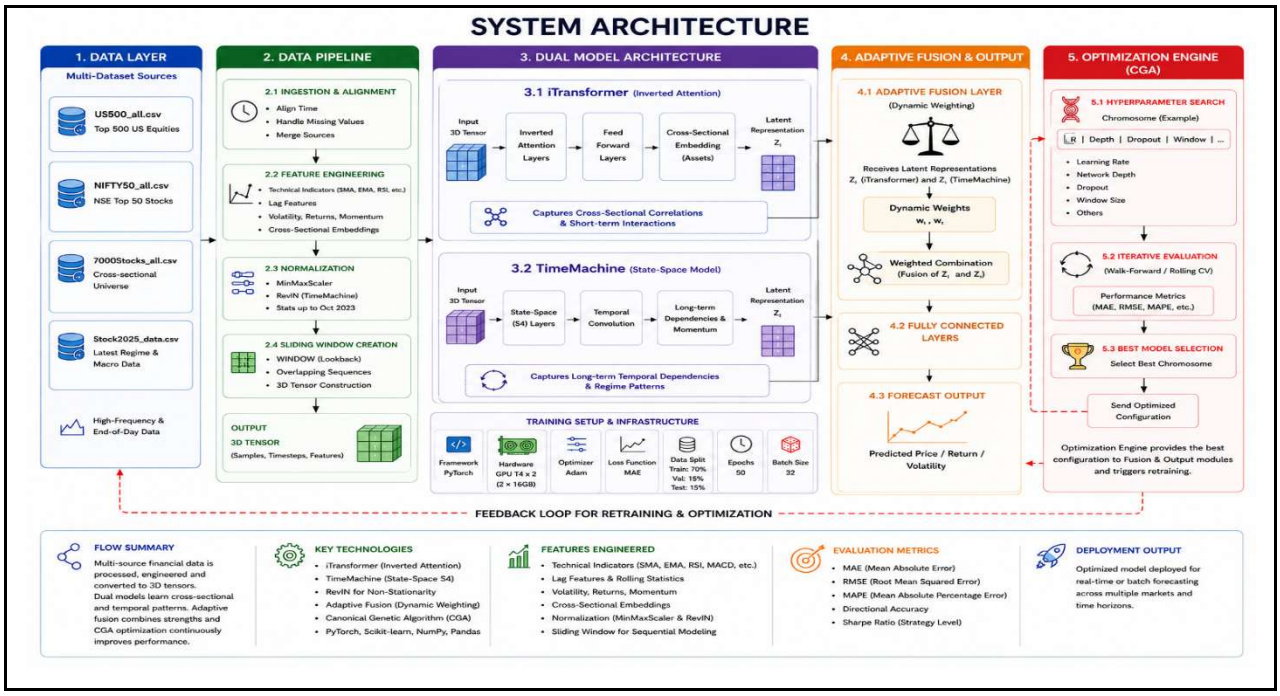


Figure 1. System Architecture of the Proposed Enterprise Financial Forecasting Engine.

3.2 Multi-Dataset Financial Data Integration

Any financial prediction mechanism would have theoretical soundness and practical feasibility entirely based on the variance, depth, and quality of its base data. In short, using a machine learning model trained on one asset fails badly, due to catastrophic overfitting and extremely poor cross-market generalization[7. As a result, the framework integrates high-frequency and end-of-day covariates from various heterogeneous sources to build a complete compendium of the multivariate matrix. The data are available on Kaggle: https://www.kaggle.com/datasets/bijoybhadra/stock-market-datasets, it contains a variety of diverse structural indices and aggregated collections:

US500_all. csv: Historical price, volume and volatility characteristics of the top 500 US equities by market capitalization providing a macroscopic view of systemic economic health and institutional capital rotation.

NIFTY50_all. csv - This dataset captures the behavior of the top stocks on India s National Stock Exchange (NSE) and is critical in introducing key aspects such as geography, regulation source and time dimension into the training corpus.

7000Stocks_all. csv: This file contains a more heterogeneous set of capitalization tiers and can serve as a huge cross-sectional data matrix mainly to train the attention heads associated with each input variable in the iTransformer in order for this model not to focus on some asset-specific idiosyncrasies but instead learn universal market microstructures.

Stock2025_data. csv: The most modern dataset containing the combination of new regime shifts, structural breaks and macroeconomic changes that are taking place in latest financial year to ensure the weights of model accurately reflect then market.

3.3 Feature Engineering and Representation Construction

The nature of financial raw data itself gives rise to vast, potentially highly disruptive scale differences for example, overall daily trading volumes often orders wider in the tens of millions and yet percentage returns (or fractional price movements) are nervously close to zero[22]. Before training the response, MinMaxScaler from sklearn scales the gradient descent trajectory is strictly normalized so that dominant high-magnitude gradients cannot stifle minor features integrated within fractional indicators. Preprocessing[6][13]. Mathematically mapping all the multi-dimensional feature vectors transforms to a limited, bounded geometric interval, uniformly weighing them for the early epochs.

In addition to standard normalization, the TimeMachine architecture purposely supports specialized stabilization via Reversible Instance Normalization (RevIN). RevIN is built upon the symmetrical addition and subtraction of instance-wise statistical moment values (mean and variance) based on data up to October 2023, which has shown to dramatically reduce severe non-stationarity challenges across long-horizon financial datasets, stabilizing training while preserving original absolute value distributions essential for high-quality forecasting[23]. The sequences, which represent features, are not fed to the models at once In contrast, an overlapping sliding window paradigm is used. This defines a WINDOW parameter for the historical lookback horizon, converting static continuous features into dynamic short-term 3D tensors with shape (samples, timesteps, features). The advantage of this topological representation is that local volatility clustering, momentum trajectories and mean-reversion patterns remain preserved and are passed to the neural network as a single coherent unit[24].

3.4 Deep Learning Experimental Setup

This physical instantiation of the neural topologies needs to be mapped accurately into specific spatial locations within the GPU memory hierarchy. Now let me explain the single streams of process that occur during an actual execution, normalized and windowed data is actually instantiated into PyTorch TensorDataset objects and are thus consumed one by one after being passed through a highly optimized DataLoader through your deep learning architectures[10]. This batched exracting reduces the terrible host-device new smart transfer insoles that afflict vast scale tensor funtions. The deep learning setup makes use of conventional loss metrics, e.g. Mean Squared Error (MSE) penalizes large deviations by outliers heavily and also Mean Absolute Error (MAE),

used to evaluate the general accuracy of the central tendency across projections[12][24]. Also, we include the R^2 variance score to mathematically measure the ratio of target variance explained by the network. To prevent catastrophic overfitting, a common problem of chaotic, extremely noisy financial modeling the environment incorporates the following features: parameterized dropout layers; extreme weight decay inside optimization functions; and clipping gradients to constrain the very sensitive selective scan mechanisms in the state-space models.

3.5 Canonical Genetic Algorithm Optimization Framework

Your experimental framework installs a latest Canonical Genetic Algorithm (GA) to traverse through the immense non-convex hyperparameter search space generated by deep learning structures. All Cross-validation or random search approaches are computationally impossible[8]. When considering minimizing millions of possible combinatorial sequences, as discussed in the literature with focus on complex neural networks hyperparameters for example: testing all potential combinations could take several years to finished iterations.

The GA optimization framework creates a very diverse and random population of candidate hyperparameter sets, in the form of digital chromosomes[11]. For example, the fitness of each individual is computed by instantiating a localized version of the forecasting model, performing an approved incomplete training cycle (e.g., FAST_MODE flag is set), and leveraging the inverse of validation loss[14]. The algorithm employs a set of evolutionary operators purposefully tailored to balance exploration and exploitation during structure searches. Proportional selection, typically performed using a roulette wheel approach, filters the best-performing configurations to become parent models. Uniform and multi-point crossover operators mix the hyperparameter genes from classes that have similar performance to produce offspring representations[11]. Simultaneously, a bounded mutation operator that may randomly change the dropout rate, window size or learning rate momentum is integrated to avoid nearby local functional optima. A survival of the fittest mechanism directly transfers winners into the following generation without any crossover, guaranteeing that optimal architectures are preserved and not lost to random mutational drift.

3.6 Evaluation, Visualization, and Deployment

Model efficiency in quantitative finance is not determined solely by numbers miscalculated. In this framework, we scan for a set of different evaluative metrics that measure distinct aspects of predictive success[25]. Meanwhile, Mean Squared Error (MSE) and Mean Absolute Error (MAE) measure the mean magnitude of errors in a set of predictions while squaring those errors to give extra weight to large deviations, an essential requirement for risk management. The Coefficient of Determination (R^2): This is used to assess the amount of variation in the financial instrument that

can be predicted by the independent features[26]. Next, the binary classification accuracy of the forecast (i.e., if prices for certain assets rise or fall) is very important as many trading strategies focus on price direction rather than absolute numerical values; this means that Aggregate Directional Accuracy (ADA). During training, we will perform visualizations comparing predicted trajectories with their ground truth values while overlaying probabilistic uncertainty bounds defined through dual-network variance estimates so that model predictions are backed by strong statistical confidence bands.

4. PROPOSED METHOD

The section present a formal mathematical exposition for the continuous-time dynamics, architectural subversions and evolutionary algorithms that allows the proposed hybrid framework to perform with state-of-the-art success across benchmark financial forecasting tasks[3]. In this paper we propose a new system that combines state-of-the-art modeling in the state-space paradigm using the TimeMachine architecture, and cross-variate representation learning in the iTransformer framework. In addition, it includes adaptive hyperparameter optimization through Canonical Genetic Algorithm (CGA) for automatically discovering the best architectural design of the multivariate stock market predictor[8]. The framework processes financial sequences based on historical OHLCV information, builds out temporal representations using sliding-window mechanisms, trains and validates models and ultimately creates explainable next-day forecasting outputs.

4.1 Proposed Model Workflow

This unified workflow pulls out clear-cut financial patterns from messy, unpredictable markets using a deep-learning pipeline. It starts with grabbing historical stock data—sometimes from massive CSV files, sometimes straight from live APIs. Once all that data is in hand, it gets cleaned up: filling in missing values, building features, standardizing, and running MinMax normalization so everything stays balanced during training[2]. To keep the models honest and avoid any peeking into the future, the workflow chops the data into training, validation, and testing sets.

Next, the system turns these processed sequences into overlapping time windows, controlled by a lookback setting. This helps the model catch both wild short-term swings and slower, longer patterns. Depending on how the architecture is set up, those windows head into either the TimeMachine State-Space forecaster or the iTransformer with its inverted attention method[1][2]. If the Genetic Algorithm module is turned on, it steps in to adjust hyperparameters on the fly for the best possible model setup.

Then, the actual training gets underway with AdamW and L2 regularization keeping the model in check. When training wraps up, predictions get transformed back to their original scale. In the end, it's not just about the numbers, the workflow uses SHAP to explain what's going on and checks how much error is left, as shown in Figure 2.

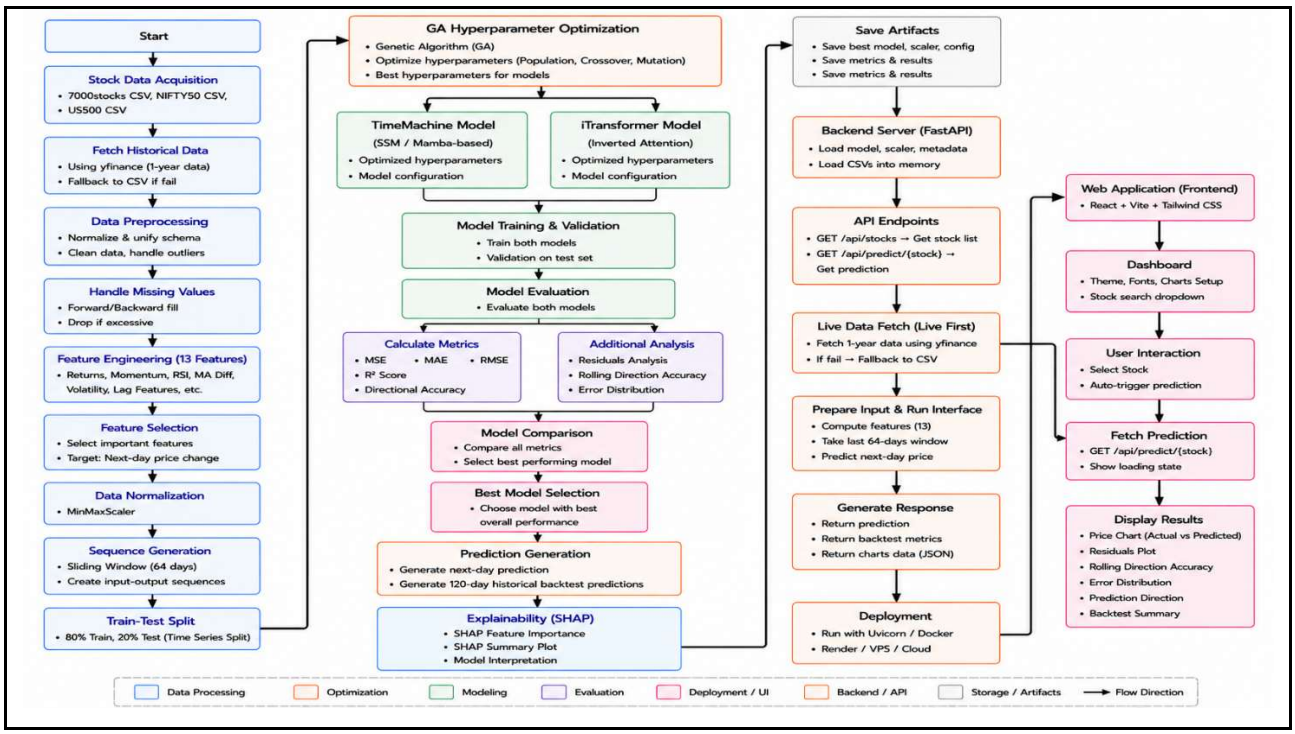


Figure 2: Proposed hybrid workflow

4.2 Hybrid Temporal Feature Representation

The stock markets don't just move at random—big shifts come from outside events like changes in the economy, while the day-to-day action follows its own patterns over time. To make sense of all this, the framework here doesn't just grab single data points. Instead, it slices the data into overlapping sequences, each one capturing a short run of the market as a whole story.

Rather than relying on simplistic point-wise embeddings, the methodology constructs overlapping sequence windows where each temporal patch represents a unified financial trajectory[12]. This sequential representation allows the model to internalize momentum transitions, volatility clustering, moving-average deviations, and directional reversals as coherent mathematical objects rather than isolated observations.

The input sequence representation is defined as:

$$X = [x_1, x_2, x_3, \dots, x_T] \in \mathbb{R}^{T \times d}$$

where:

- T denotes the Temporal sequence length,
- d denotes the number of engineered financial features.

To keep results fair across different stocks or other assets with wildly different price scales, it uses normalization tools like MinMaxScaler and reversible instance normalization (RevIN)[14]. With these, the model isn't just learning to match exact price levels. Instead, it's tuned in to the shapes and patterns over time the underlying moves regardless of the market's absolute numbers.

The normalization process is expressed as:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

This representation significantly enhances cross-asset generalization and stabilizes deep learning optimization.

4.3 TimeMachine State-Space Forecasting Mechanism

The TimeMachine architecture, based on the Mamba selective state-space framework, models financial time series as observable realizations of latent dynamical states that continuously evolve[2]. The model does not predict a sequence with the traditional recurrent architectures but rather, instead it specifies sequential forecasting in terms of continuous-time differential equations.

The continuous state-space formulation is defined as:

$$h'(t) = Ah(t) + Bx(t), \;\; y(t) = Ch(t)$$

where:

- $x(t)$ represents the multidimensional financial input sequence,
- $h(t)$ denotes the latent hidden state,
- A, B, C are learnable transition matrices,
- $y(t)$ corresponds to the generated output prediction.

To process discrete financial observations, the continuous equations are discretized using Zero-Order Hold (ZOH) transformations:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t$$

Selective parameterization, in which the state transition dynamics are an adaptive function of the current market input, is also integrated into the TimeMachine framework[4]. Thus, the architecture uses a tailored approach by blowing up important financial occurrences and down-playing unimportant market variations.

In contrast to the global scale algorithm comprising Transformer architectures (quadratic complexity), our selective state-space mechanism achieves linear computational scaling:

$$\mathcal{O}(n) \text{vs} \mathcal{O}(n^2)$$

This characteristic allow us to efficiently forecast long-horizons on extremely large-scale financial datasets with very low memory requirements[10].

4.4 iTransformer Sequential Forecasting

iTransformer framework with an inverse self-attention mechanism for the multi-variant time-series forecasting which aims to overcome the so-called "Race Transformer Paradox". Specifically, traditional Transformers use the tokenization feature across the temporal dimension for sequential data, thus compelling heterogeneous financial indicators into a common embedding space[3].

Propose a new paradigm: iTransformer, that treats each variate as an independent token to reverse this. The embedded transposed financial tensor as:

$$Z = X^T W_e$$

where:

- X^T denotes the transposed input matrix,
- W_e represents the learnable embedding projection.

The attention operation is subsequently applied across variates rather than across time:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

where:

- Q, K, and V correspond to Query, Key, and Value matrices,
- d denotes hidden dimensionality.

By making this attention focus upside down, the architecture can learn cross-asset correlations along with similar sector-wise dependencies explicitly while minimizing interaction noise by disregarding having different feature scales[9].

The framework enhances training stability through independent Layer Normalization on each variate representation making it robust towards convergence and generalisation performance.

4.5 GA-Based Hyperparameter Optimization

This results in the simultaneous use of selective state-space network and inverted attention architectures, yielding extremely non-convex optimization space. To cover this search space effectively, the framework uses a Canonical Genetic Algorithm (CGA), inspired by biology's evolutionary principles[8].

Each candidate architecture is encoded as a chromosome:

$$C = [g_1, g_2, g_3, \dots, g_n]$$

where each gene g_i corresponds such as:

- learning rate,
- hidden dimensions,
- attention heads,
- mutation rate,
- dropout ratio,
- batch size,
- sequence window length.

The fitness of each chromosome is computed using inverse validation loss:

$$Fitness = \frac{1}{Loss + \epsilon}$$

where:

- $Loss$ denotes validation error,
- ϵ is a small stability constant preventing division by zero.

Mechanisms for tournament selection identify candidates for reproduction who are high performing. Uniform crossover exchanges one or more structural characteristics between parent solutions, and Gaussian mutation introduces controlled stochastic perturbations to preclude premature convergence[11].

The Evolutionary Optimization Process determines forecasting performance across multiple generations, iteratively minimizing various prediction error metrics like RMSE, MAE and MSE.

4.6 Comparative Forecasting and Model Selection

In this case, the proposed framework includes a comparative forecasting layer that can dynamically measure and adapt to the varying nature of how well the TimeMachine and iTransformer architectures would perform compared to each other as market conditions change[10].

The iTransformer architecture excels in extreme multivariate environments with high levels of cross-asset interaction and non-linear dependencies[12]. On the other hand, for long time forecasting task with deep temporal memory and low computational latency requirement, the TimeMachine selective state-space framework is implemented very effectively[15].

Model performance is evaluated using multiple statistical metrics including:

- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)

- Coefficient of Determination (R^2)
- Directional Accuracy

The RMSE evaluation criterion is defined as:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

where:

- y_i denotes actual stock prices,
- $\hat{y}_i$ represents predicted values,
- n is the number of observations.

The intelligent selection layer is able to automatically select an optimal forecasting architecture based on the dimensionality of your dataset, the volatility structure, and predictive stability[22]. Moreover, ensemble aggregation mechanisms may consider combining outputs from both paradigms to yield more smooth and robust forecasting trajectories.

5. METHODOLOGY

Proposed model is a dual deep learning framework that has been especially designed for multi-dataset stock prices forecasting. The following advanced technologies are integrated by the framework to provide its analytical capabilities: TimeMachine(Mamba/SSM), iTransformer, Genetic Algorithms, FastAPI and Chart. js.

5.1 Data Collection and Standardisation

The section describes the ingestion and alignment of disparate financial datasets necessary for stress testing the predictive model across a range of global markets. The integration of datasets archives from Kaggle: https://www.kaggle.com/datasets/bijoybhadra/stock-market-datasets, it contain four CSV but the proposed system incorporated with following three: 7000Stocks_all.csv (global equities), NIFTY50_all.csv (for Indian NSE constituents), and US500_all.csv (S&P 500). On the other hand, every source has different column names and date_formats and also price representations, therefore having a dedicated normalise_dataset() function unifies all three into a single schema: Date, Stock, Open, High, Low, Close, and Volume.

The shelf does not start to model with raw data: date strings are parsed using dataset specific format directives (%Y-%m-%d for global and NIFTY data; %m-%d-%Y for US500), OHLCV columns are cast to numeric types, and rows missing a valid date, ticker or closing price are dropped. TOP_N_PER_DATASET = 40 It retains the top 40 stocks for each dataset by record count, resulting

in a combination of multi-market corpora spanning different scales of pricing, regimes of volatility and trading calendars, all chosen to stress-test model generalisation[23].

5.2 Exploratory Data Analysis (EDA)

To An automated diagnostic phase precedes any action in the tensor pipeline to ensure statistical coherence of the data and drive loss-function parameterization. Before any tensor operations however, an automated diagnostic phase runs against each dataset independently controlled by the RUN_EDA flag. Three analyses are produced:

Mean, standard deviation, and quartile summaries of the Close column to surface scale anomalies or distributional discrepancies between markets. A histogram with superimposed kernel density for each dataset, with the corresponding sign for loss-function weighting is plotted (up to 2,000 rows per dataset)[26]. Temporal sequences of close-prices distributions for the three most-recorded stocks per dataset (down-sampled over 2000) are plotted to highlight structural breaks, between-regime trends and out-of-greens trading times[18].

These outputs corroborate that the normalisation steps operate on coherent distributions and that no single dataset pathologically dominates the combined training corpus.

5.3 Preprocessing, Feature Engineering, and Sequence Construction

The process of preparing the financial data for deep learning consists of a few main steps like splitting by timestamp (chronological split), normalisation per stock, target encoding and structure features into temporal sequence etc[26].

At the 80% date of the combined corpus a strict 80 / 20 chronological split is applied (SPLIT_FRACTION = 0.80). The data split is computed from the data dynamically, so it is reproducible and not dependent on which stocks survive filtering. all scalers of features are fit only on the training partition data and applied on test partition using same learnt parameters, completely preventing any look-ahead bias.

A MinMaxScaler is fitted separately on the training closing prices at each stock and then stores in a scalers dictionary, to use during inference time. If you fit a single global scaler across all stocks then high-value equities will dominate the scaled space as well -- per-stock normalisation keeps the gradient quality consistent across the entire price range[27]. Log returns are calculated as $r_t = ln(C_t/C_{t-1})$ and Z-score normalised using training-partition statistics, lastly they are clipped to [−5, +5] to suppress the crash spikes.

This leaves us with 13 input channels as 11 features are derived from the scaled close price and normalised return series. Min_periods = 2 is used for all rolling operations to maximise the retention of data on the boundary of each stocks historical horizon. Rationale: Before the tensor is

assembled, all NaN and infinite values are replaced with a zero. The corresponding calculations and economic justifications for these channels are provided in Table 2 below.

Table 2: Technical Feature Engineering

Feature (13 channels)	Formula & Computation	Economic Rationale
Close_Scaled	MinMaxScaler(Close) ∈ [0,1] (Fit on training partition only)	Normalised price level; eliminates scale disparity across stocks
Return_Scaled	clip($(ln\ (C_t/C_{t-1}) - \mu)/\sigma, -5, +5$) ($\mu$, σ from training partition)	Stationary momentum; Z-score bounded to suppress crash spikes
Momentum_3 / 5 / 10	Close_Scaled_t−Close_Scaled_{t−n} (n = 3, 5, 10 days)	Short, medium, and intermediate price drift
Return_Mean_5 / 10	Rolling mean of Return_Scaled (window = 5, 10; min_periods = 2)	Trend persistence and directional bias
Return_Std_5 / 10	Rolling std of Return_Scaled (window = 5, 10; min_periods = 2)	Local volatility estimate; rises during turbulent periods
Price_MA_Diff_5 / 10 / 20	Close_Scaled − MA(n) (n = 5, 10, 20 days; min_periods = 2)	Mean-reversion signal; positive = price above trend
RSI_14	(100 − 100 / (1 + avg_gain / avg_loss)) / 100 (14-day window ⟶ ∈ [0,1])	Momentum oscillator; values near 1 or 0 signal overbought / oversold

The model predicts a normalised price delta rather than an absolute next-day price:

$$y_t = (Closed_{Scaled_t} - Closed_{Scaled_{t-1}})/DIRECTION\_SCALE$$

As a target, DIRECTION_SCALE is the median (0.006528) of all non-zero single-day moves across the entire training corpus and as such, it's unitless in direction since it stays constant nearly regardless of price level. Delta targets are capped at ±5.0 Then finally at inference time get the prediction decoded: predicted_close = scaler. inverse_transform(clip(last_Close_Scaled + pred_delta × DIRECTION_SCALE, 0, 1))

By employing a one-day stride, the preprocessed feature matrices are partitioned into multiple overlapping input tensors. Each sample $X_t \in \mathbb{R}^{64\times13}$ contains the data from the 64 trading days prior to day t (WINDOW = 64), so that we can include short-term momentum, medium-term

trend, and early mean-reversion signals in a single feature. Samples are wrapped in cases of TensorDataset and loaded with DataLoader with batch sizes of 256 (GPU) or 128 (CPU).

5.4 Model Architectures and Hyperparameter Optimisation

The section discuss the two main neural architectures used in our framework, namely TimeMachine and iTransformer, and present how a genetic algorithm finds their optimal structural parameters.

The Mamba selectiveState Space Model (SSM): The TimeMachine architecture: A new architecture for multivariate financialtime-series forecasting. There are four sequential stages in the forward propagation process. The first is RevIN (Reversible Instance Normalization), which normalizes each 64×13 input window independently, while storing the statistical parameters for denormalizing later[2][3]. Second, a linear feature projection transforms the 13-dimensional feature space into a higher-dimensional latent representation d_{model}. Third, several parsed Mixture of Experts MambaBlocks iteratively operate on the temporal representation through sequences of RMSNorm → depthwise Conv1D to learn local context $(dconv = 4)$→ gating by SiLU and optimized scan in TorchScript, One layer of the recurrent structure could be expressed as:

$$h_t = dA \cdot h_{t-1} + dB \cdot x_t.$$

This makes it possible to model dependencies on each element in the long-sequence, and it is followed by residual projection layers that are repeated num_layers times. Then the processed sequence is flattened and fed through a linear projection head, to get the output $pred\_delta$ (the expected change in price on next day).

The iTransformer utilizes self-attention on the feature dimension rather than time and helps to learn cross-indicator correlations (e.g. RSI vs. momentum) which temporal attention would no longer be able to capture efficiently[5]. RevIN normalisation produces a linear layer which maps the 64-timestep sequence of each feature channel to a d_model-dimensional token. The feature tokens are processed by a standard TransformerEncoder (e_layers layers, 10% dropout, n_heads auto-selected as the largest divisor of d_model from {8, 4, 2, 1}), a final linear head projects the output of the first token to pred_delta.

Canonical Genetic Algorithm is utilize to optimize forecasting performance, the framework optionally employs when $RUN\_GA = True$. The GA independently searches three critical hyperparameters for each deep-learning architecture: learning rate $(lr \in \{10^{-4}, 3 \times 10^{-4}, 5 \times 10^{-4}, 10^{-3}\})$, hidden representation size $(d_{model} \in \{32,64,128\})$, and network depth $(num\_layers \in \{1,2,3\})$. This search space generates 36 candidate configurations for each

model[8]. Every candidate undergoes lightweight probe training for 2 epochs using 40 mini-batches, followed by validation on 30 batches to estimate generalization capability.

The optimization objective is governed by the fitness function:

$$Fitness = \frac{1}{val\_price\_loss + 0.01 \times (1 - dir\_accuracy)}$$

This formulation simultaneously minimizes reconstruction error while rewarding directional prediction accuracy. Evolution across generations is achieved through tournament selection, per-parameter random-parent crossover, and a mutation probability of 25%, enabling efficient exploration of the hyperparameter space[11]. When $RUN\_GA = False$, predefined baseline configurations are directly applied. In this setup, TimeMachine uses $d_{model} = 32$with $num\_layers = 2$, whereas iTransformer uses $d_{model} = 64$and $num\_layers = 2$.

5.5 Training, Evaluation, and Model Selection

During the training and evaluation pipeline of forecasting models, to extract the one with maximum performance. We use the AdamW optimizer with weight decay regularization and GA-optimized learning rates for both TimeMachine and iTransformer. Which makes the overall idea of training objective more robust with not only delta prediction loss but also having both price reconstruction and directional classification losses to enhance accuracy in both numerical values and market movement directions[17]. The training process uses gradient clipping for stabilization and then it applies a ReduceLROnPlateau scheduler, which reduces the learning rate when validation performance has stopped improving. Models are automatically checkpointed, with the best still being saved during training[19]. You can resume a previous training by loading back from example checkpoints. They stop training after FAST_MODE up to 15 epochs or full mode 20.

Then, the models are evaluated on the test dataset with RMSE, MSE, MAE, R2R^2R2 and directional accuracy metrics. When selecting the final model, a weighted composite scoring approach is employed that gives priority to RMSE, MAE and directional accuracy[25]. The better model is saved with architecture details, hyperparameters, feature configuration and scaling parameters. Furthermore, if params.RUN_SHAP = TrueRUN_SHAP = TrueRUN_SHAP=True, then Kernel SHAP analysis is conducted on the chosen test samples to quantify each input feature channel's contribution for providing post-hoc interpretability of the financial analysis.

5.6 Deployment and Interactive Forecasting Dashboard

For production deployment, it is implemented as containerizable FastAPI microservice on port 8000. Backend will load financial CSV archives (back end initialization) and read best_model_info. json, Loads the best architecture with its saved hyperparameters and sets the model

to evaluation mode. A three-level fallback mechanism leverages robustness: the preferred model gets loaded first, then checkpoints that are available at saved_models/ get scanned for validity and finally if there is no valid model an error response in a structured format will be returned.

On each /api/predict/{stock} request, the backend first tries to fetch 12 months of live market data from yfinance. Insufficient records, system quietly elects to fallback to the local CSV and flags as offline. The same feature-engineering pipeline applied at training phase is now performed — finally the last window of 64 days run to predict next day price and percent change. On top of that, we carry out a 120-day sliding-window backtest to calculate statistics for your predictions in the past and their directional accuracy before returning that as a JSON response.

The frontend is implemented with HTML, CSS and JavaScript, which loads available stocks dynamically on the page then allows selection using search from options for predctions that will be presented as summary cards include five interactive Chart.js panels, as illustrate in Table 3. It features an responsive dark-theme in the background, optimized chart rendering, automatic removal of empty canvases and layouts that adapt to every screen size.

Table 3: Diagnostic Chart Panels

Chart	**Type**	**Purpose**
Actual vs Predicted Price	Dual Line	Compares real closing prices with predicted prices over the 120-day backtest period to identify prediction lag and bias.
Residual Error & Price Movement	Bar + Line	Visualizes daily prediction error alongside actual and predicted price movement trends.
Rolling Direction Accuracy	Area Line	Shows 10-day rolling directional accuracy against a 50% baseline to evaluate predictive reliability.
Prediction Error Density	Histogram	Displays the distribution of residual errors to identify forecast bias and variance.
Actual vs Predicted Direction	Grouped Bars	Compares predicted and actual market direction on a day-to-day basis to highlight classification performance.

6. RESULT & DISCUSSIONS

The experimental dataset include three sources in markets, 7000Stocks, NIFTY50 and US500. We then filtered for high-coverage securities, yielding a final pure CSV dataset of 211,560 rows across 120 stocks (40 from each selected dataset). The three sources have non uniform price scales and dispersion patterns, evidenced by the distributions of their closing-prices. Long right tails within a wide price range: The 7000Stocks and US500 samples are largely concentrated on the left of the

price spectrum but exhibit long right tails, with NIFTY50 having quite a broad high-price boundary due to large low velocity Indian equities, as demonstrated in Figure 3, 5, & 7. This justifies per-stock scaling and feature normalization before the training process.

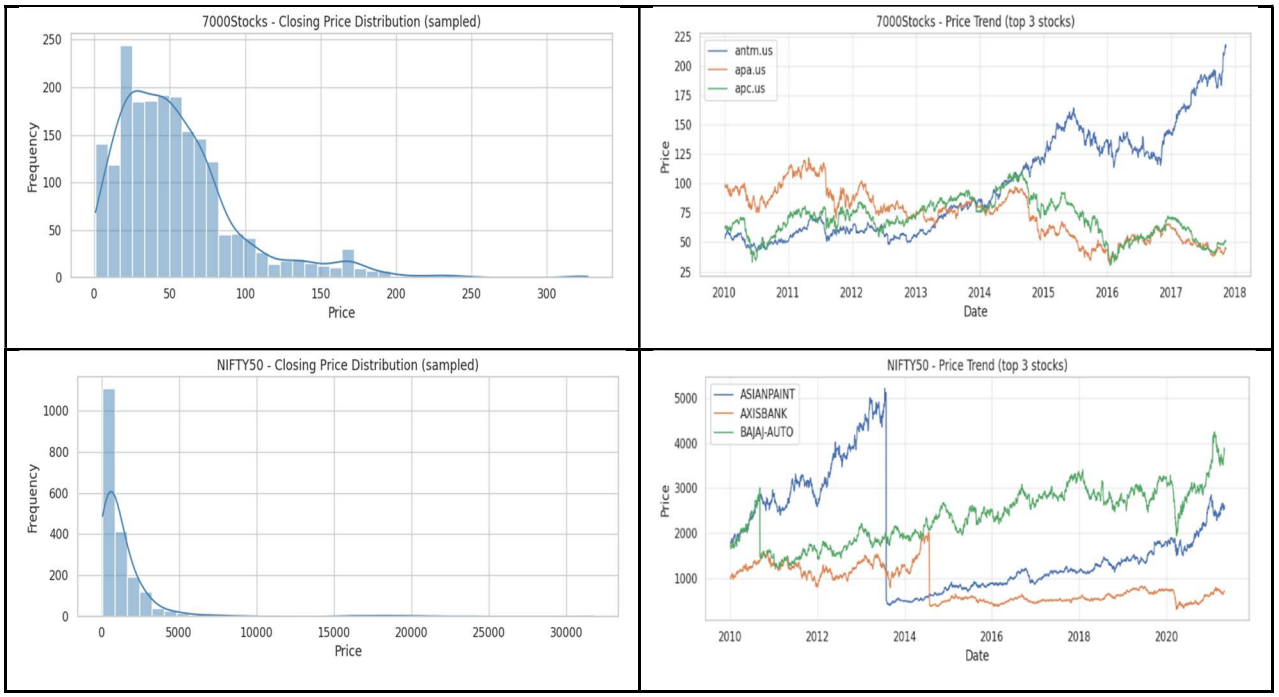
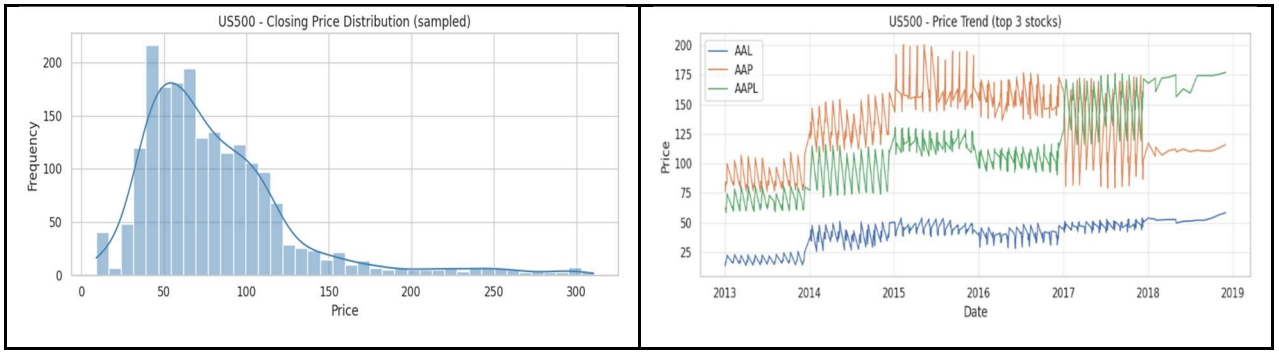


Figure 3: Distribution of Closing Prices in the 7000Stocks Dataset., Figure 4: Historical Price Trends of Selected Stocks from the 7000Stocks Dataset., Figure 5: Distribution of Closing Prices in the NIFTY50 Dataset., & Figure 6: Historical Price Trends of Selected Stocks from the NIFTY50 Dataset.

The time-series trend plots also demonstrate the suitability of a sequence model for this task. Figure 4 displaying the growth & decline of stocks (2010-2018) from the sample of 7000Stocks Sensitivity of NIFTY50 stocks to long history and reinstated non-linear properties with closer sharp corrections and recoveries (Figure 6). Figure 8 plots price jumps and a periodic pattern of sampled US500 samples with black, white, red dots representing the top plotted securities. The combined high-coverage stock trend further confirms that the model must handle multiple assets with different price levels, volatility regimes, and temporal lengths (Figure 9).



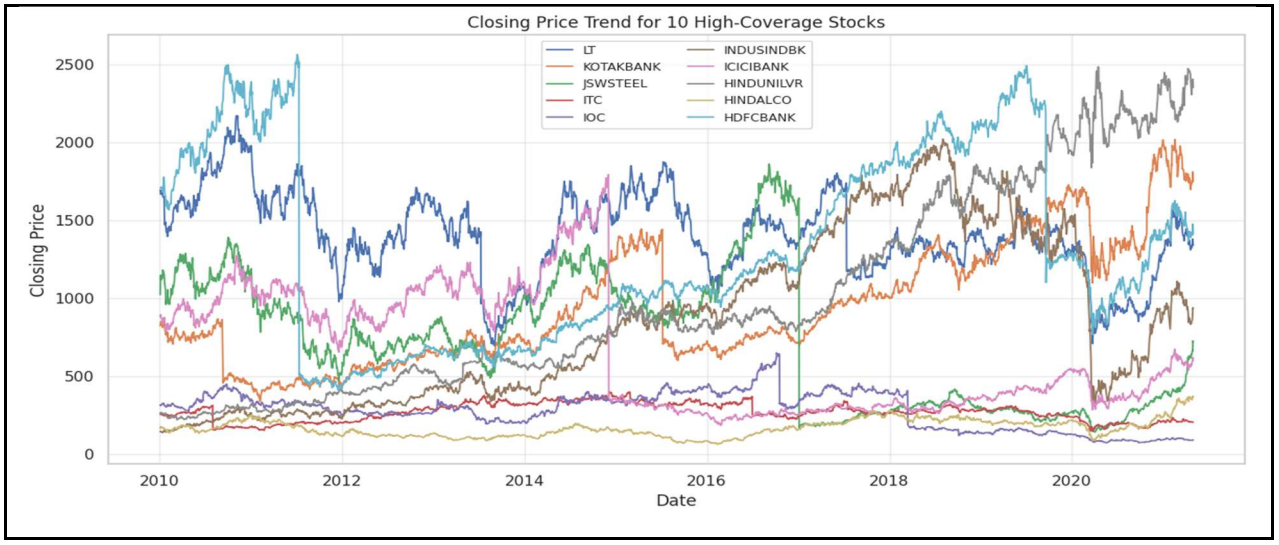


Figure 7: Distribution of Closing Prices in the US500 Dataset., Figure 8: Historical Price Trends of Selected Stocks from the US500 Dataset., & Figure 9: Comparative Multi-Stock Price Trends Across Datasets.

Table 4: Dataset Closing-Price Summary

Dataset	Rows	Stocks	Mean Close	Std. Dev.	Min	25%	Median	75%	Max
7000Stocks	79,200	40	55.5325	41.4215	0.56	25.562	48.81	72.0842	335.57
NIFTY50	1,12,400	40	1666.09	3343.2	60	349.55	721.025	1614.55	32862
US500	19,960	40	83.1321	51.4297	8.75	49.1862	69.815	101.43	336.64

The characteristics of the dataset are summarized in Table 4 with mean closing price over the period of 55.5325 and maximum closing price of 335.5700 for the 7000Stocks dataset, while US500 has similar maximum but much higher mean 83.1321. While NIFTY50 is wider, with mean 1646.0903 and maximum 32861.9500. These results provide rationale for the use of MinMax scaling for close prices and indicators based on return in normalized values as preprocessing step.

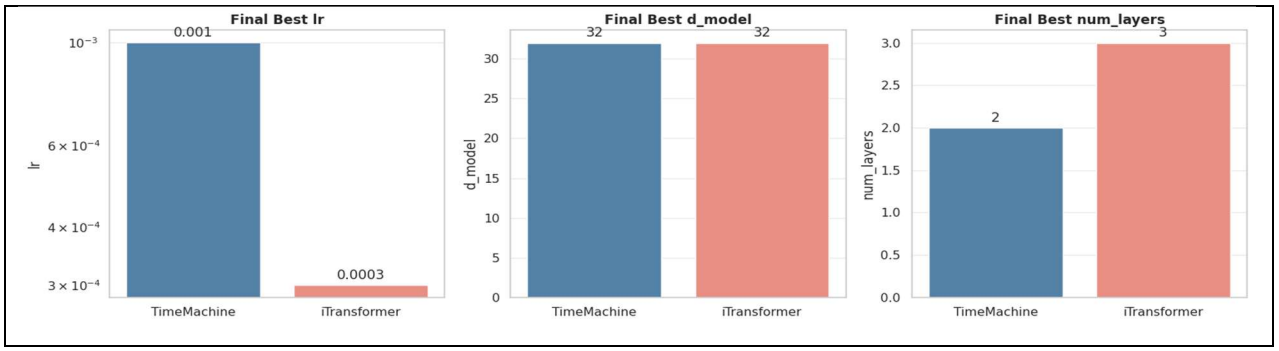
The notebook went with a chronological split instead of picking samples at random, which makes sense for forecasting since it keeps data from the future out of the training set and avoids leakage. The split date was set as 15 Nov 2018, so the training set covered about 80% of the timeline. For predictions, the model used a 64-day input window to forecast the next day's residual. Table 5 lays out how sequences were generated and how the data loader was configured. The features included scaled closing prices, scaled returns, various momentum indicators, rolling mean and standard deviation of returns, moving average price gaps, and RSI.

Table 5: Preprocessing and Data Loader Summary

Item	Notebook Output
Final rows	2,11,560
Unique stocks	120
Train sequences	1,79,520
Test sequences	21,760
Input window	64 days
Number of input features	13
Train stocks with sequences	120
Test stocks with sequences	40
Direction scale	0.006522
Target mode	Delta / residual prediction
Train batches	702
Test batches	85

The models are trained with 13 input features: Close_Scaled, Return_Scaled, Momentum_3, Momentum_5, Momentum_10, Return_Mean_5, Return_Mean_10, Return_Std_5, Return_Std_10, Price_MA_Diff_5, Price_MA_Diff_10, Price_MA_Diff_20, and RSI_14. This feature setup lets the models pick up on short-term lag patterns as well as signals from technical momentum and volatility.

The model used a genetic algorithm to tune the main hyperparameters for both TimeMachine and iTransformer. Our search focused on the learning rate, hidden dimension (d_model), and the number of layers. In the end, the best setup picked a higher learning rate for TimeMachine, while iTransformer performed better with a smaller learning rate but more layers (see Table 6). You can see this summarized in the parameter comparison chart in Figure 10. The GA fitness convergence plot tells a clear story: both models hit their peak fitness early on and held steady through all four generations (see Figure 11). TimeMachine's best fitness stuck at 197.511928 every time, and iTransformer held at 197.207286. Where you see more movement is in the average fitness—iTransformer's numbers got a decent bump, climbing from 191.892965 to 194.601530. TimeMachine, on the other hand, barely moved at all.



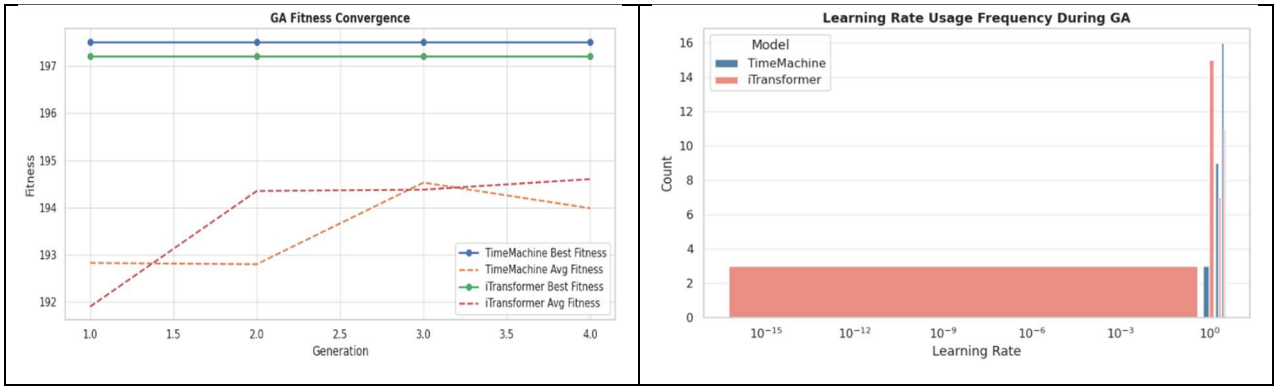


Figure 10: Final Optimized Hyperparameters Obtained Through Genetic Algorithm., Figure 11: Genetic Algorithm Fitness Convergence Across Generations., & Figure 12: Learning Rate Selection Frequency During Genetic Optimization.

Table 6: Final Hyperparameters Selected by Genetic Algorithm

Model	Gen.	Best Fitness	Avg. Fitness	Learning Rate	d_model	Layers
TimeMachine	1	197.512	192.821	0.001	32	2
TimeMachine	2	197.512	192.795	0.001	32	2
TimeMachine	3	197.512	194.528	0.001	32	2
TimeMachine	4	197.512	193.984	0.001	32	2
iTransformer	1	197.207	191.893	0.0003	32	3
iTransformer	2	197.207	194.349	0.0003	32	3
iTransformer	3	197.207	194.378	0.0003	32	3
iTransformer	4	197.207	194.602	0.0003	32	3

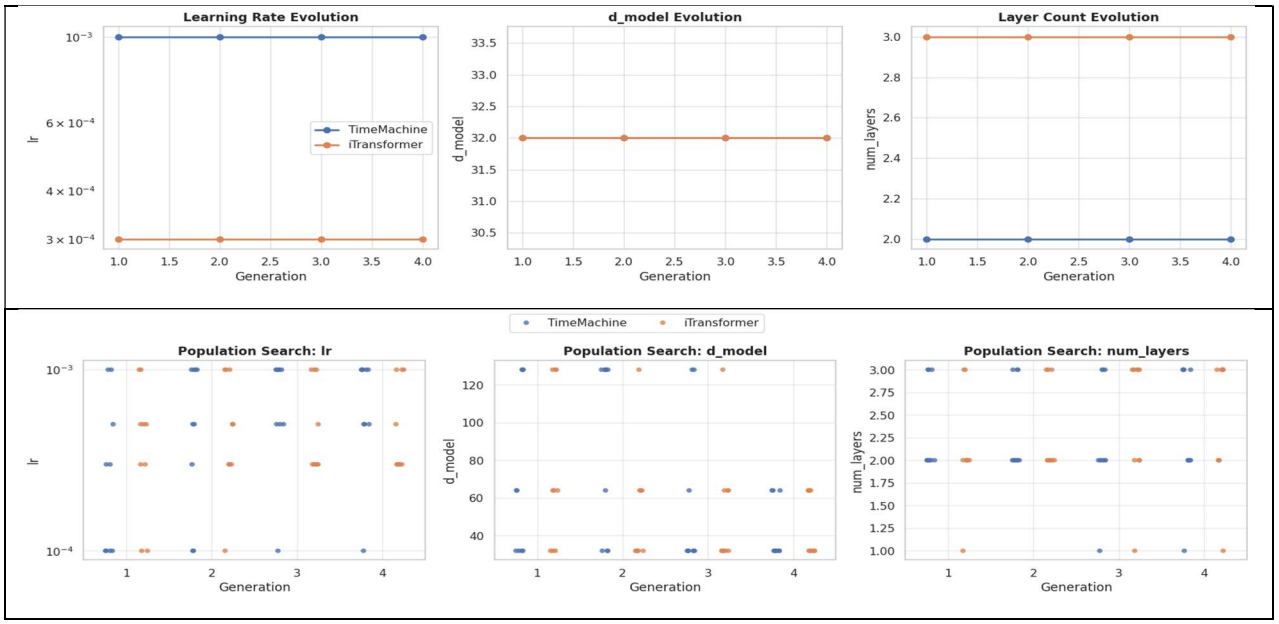


Figure 13: Evolution of Hyperparameters Across Genetic Algorithm Generations., & Figure 14: Population-Wide Hyperparameter Search Distribution During GA Optimization.

Looking at the learning-rate usage, the search wandered through quite a few values, but the best setups actually settled at 0.0010 for TimeMachine and 0.0003 for iTransformer (Figure 12). If you check out the parameter evolution and population-search plots, you'll spot that both models were fine with d_model=32, though they didn't agree on the best depth—each architecture landed somewhere else (see Figures 13 and 14).

The full training phase trained TimeMachine and iTransformer for 15 epochs each. TimeMachine used 22,459 trainable parameters, while iTransformer used 414,651 parameters. The training and validation curves show that training loss decreased steadily for both models, but validation loss remained comparatively flat after the early epochs (Figure 15). This suggests that the models learned useful short-term price reconstruction patterns, but additional epochs did not significantly improve validation generalization.

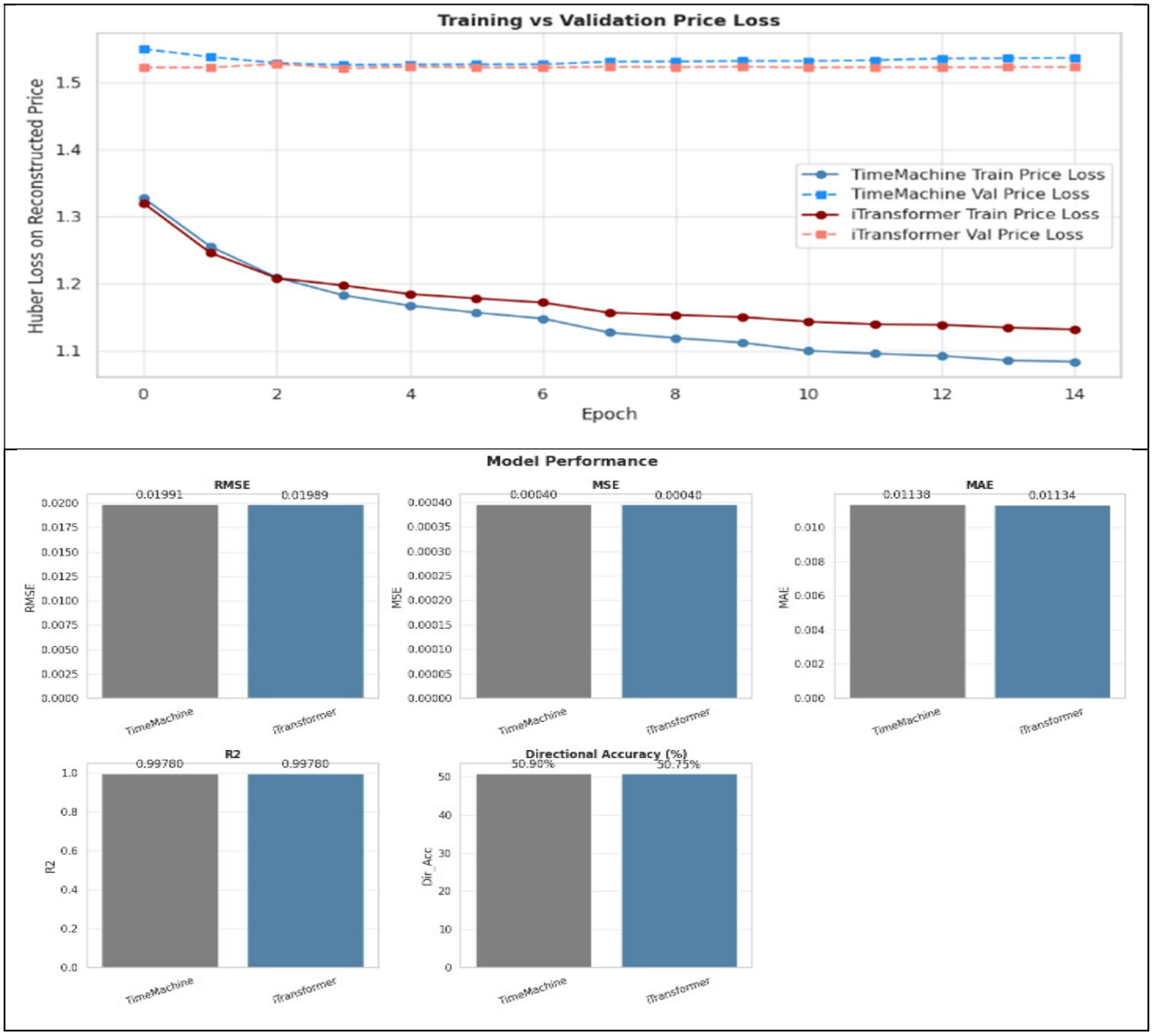


Figure 15: Training and Validation Loss Convergence During Model Training., & Figure 16: Comparative Performance Evaluation of TimeMachine and iTransformer

Table 7: Models Training and Comparison

Model	Parameters	Best Val Loss	RMSE	MSE	MAE	R2	Dir. Acc. (%)
TimeMachine	22,459	1.52558	0.01991	0.0004	0.01138	0.9978	50.9037
iTransformer	4,14,651	1.52075	0.01989	0.0004	0.01134	0.9978	50.7474

The model-performance chart and the model comparison table show that iTransformer achieved the best price-regression results across RMSE, MSE, MAE, and R2, while TimeMachine achieved slightly better directional accuracy (Figure 16 and Table 7). The final composite score selected iTransformer because it had stronger overall regression performance, with a composite score of 0.9250 compared with 0.0750 for TimeMachine.

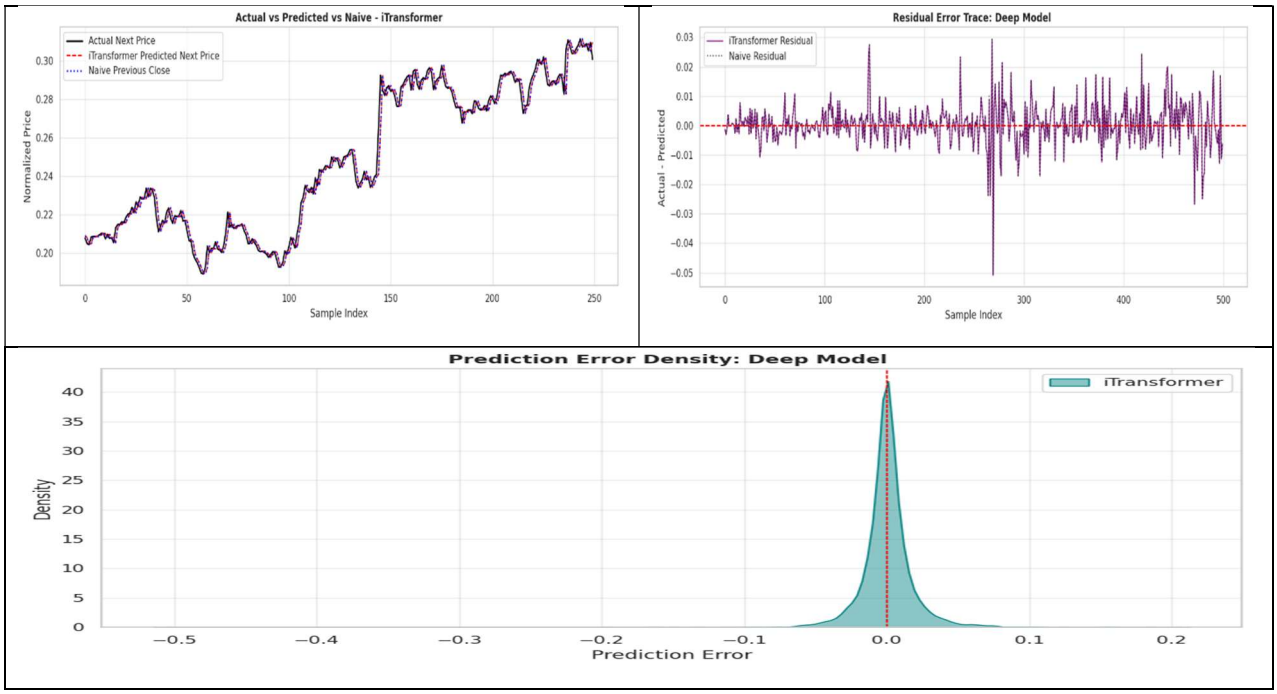


Figure 17: Actual vs Predicted Stock Price Comparison., Figure 18: Residual Error Analysis of Model Predictions., & Figure 19: Distribution of Prediction Errors Across the Test Dataset.

The price-prediction plot shows that iTransformer closely follows the actual normalized next price and remains near the naive previous-close baseline (Figure 17). This is expected in short-horizon stock forecasting, where next-day close prices are highly autocorrelated. The residual trace is centered near zero for most samples, with only a few larger spikes (Figure 18), and the prediction-error density is sharply concentrated around zero (Figure 19). These plots support the quantitative result that regression error is low.

However, directional prediction remains challenging. The rolling directional accuracy fluctuates around the 50% random-guess baseline, with occasional short windows rising above 0.60 and falling below 0.40 (Figure 20).

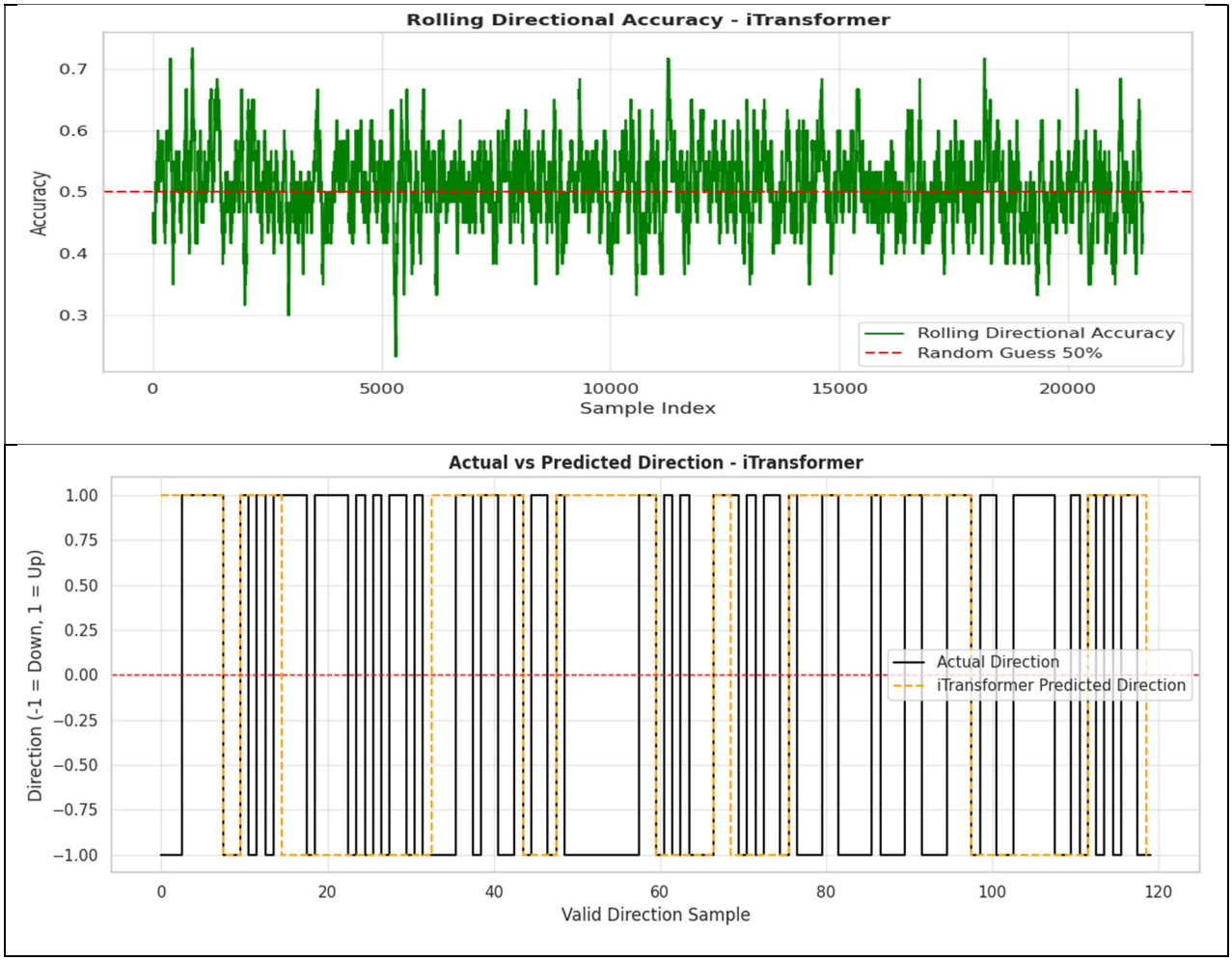


Figure 20: Rolling Directional Accuracy Over the Evaluation Window., & Figure 21: Actual vs Predicted Market Direction Classification.

The actual-versus-predicted direction plot also shows frequent mismatches between up/down movements (Figure 21). Therefore, the model is more reliable for estimating next-price level than for consistently predicting the direction of daily movement.

Table 8: Directional Classification Metrics

Model	Accuracy	Precision	Recall	F1 Score
TimeMachine	0.509	0.5111	0.7041	0.5923
iTransformer	0.5075	0.5118	0.5967	0.551

The directional metrics in Table 8 reinforce the same conclusion. Both models are only slightly above 50% accuracy, which is typical for next-day market-direction prediction under noisy conditions. TimeMachine produced higher recall and F1 score, but iTransformer was selected because the main objective prioritized accurate price forecasting with directional accuracy as a secondary component.

The selected iTransformer model was also used to generate one-to-five-business-day forecasts for individual NIFTY50 stocks. The forecast values in Table 9 are close to the last observed

closing prices, reflecting the model's conservative residual-prediction behaviour. This is consistent with the actual-versus-predicted price plot, where the model follows recent price levels closely (Figure 14).

Table 9: Model's Sample Future Forecasts

Stock	Last Date	Last Close	1-Day Forecast	1-Day Change (%)	5-Day Forecast	5-Day Change (%)	Model
ASIANPAINT	2021-04-30	2536.4	2536.21	-0.0076	2535.84	-0.0219	iTransformer
AXISBANK	2021-04-30	714.9	714.731	-0.0236	714.624	-0.0386	iTransformer
BAJAJ-AUTO	2021-04-30	3833.75	3833.67	-0.002	3834.06	0.008	iTransformer
BRITANNIA	2021-04-30	3449	3449.77	0.0224	3452.54	0.1027	iTransformer
EICHERMOT	2021-04-30	2421.65	2419.19	-0.1014	2410.5	-0.4604	iTransformer
TATAMOTORS	2021-04-30	293.85	293.978	0.0435	294.551	0.2386	iTransformer
WIPRO	2021-04-30	492.75	492.986	0.0479	493.321	0.1158	iTransformer

The sample forecasts indicate that most predicted changes are small in percentage terms. EICHERMOT shows the largest negative five-day change among the shown examples, while TATAMOTORS and WIPRO show modest positive five-day changes. This behaviour is reasonable for a next-day residual model extended iteratively over short horizons, because each step updates the projected close gradually.

SHAP analysis was applied to explain the selected model's predictions. The notebook generated SHAP values for 100 samples with 832 flattened sequence features, and the top feature was Close_Scaled_t-1. Table 10 and Table 11 confirm that the model relies most heavily on recent closing-price information, especially the immediately previous day and nearby lagged closes.

Table 10. SHAP Channel Importance & Top Features

Feature Channel	Mean Absolute SHAP	Feature	Mean Absolute SHAP
Close_Scaled	6.1853	Close_Scaled_t-1	4.11983
Return_Std_10	0.12202	Close_Scaled_t-8	1.03811
RSI_14	0.11889	Close_Scaled_t-9	0.55486
Price_MA_Diff_5	0.11557	Close_Scaled_t-7	0.14005
Momentum_10	0.11145	Close_Scaled_t-33	0.04661
Momentum_5	0.10542	Close_Scaled_t-34	0.02273
Return_Std_5	0.10469	Close_Scaled_t-39	0.0207
Price_MA_Diff_20	0.10326	Close_Scaled_t-25	0.02029
Return_Mean_5	0.10124	Close_Scaled_t-50	0.01954

Price_MA_Diff_10	0.08304	Close_Scaled_t-24	0.01881
Momentum_3	0.081	Close_Scaled_t-45	0.0139
Return_Scaled	0.07658	Momentum_10_t-14	0.01097
Return_Mean_10	0.06432	Close_Scaled_t-60	0.01062

The SHAP waterfall plot demonstrates how individual lag features push a sample prediction above or below the expected model output (Figure 22). In the shown example, Close_Scaled_t-1 contributes the strongest negative effect, while older close lags such as Close_Scaled_t-8 and Close_Scaled_t-9 contribute positive effects. The SHAP feature-distribution plot confirms that recent scaled close values dominate the model output across samples (Figure 23).

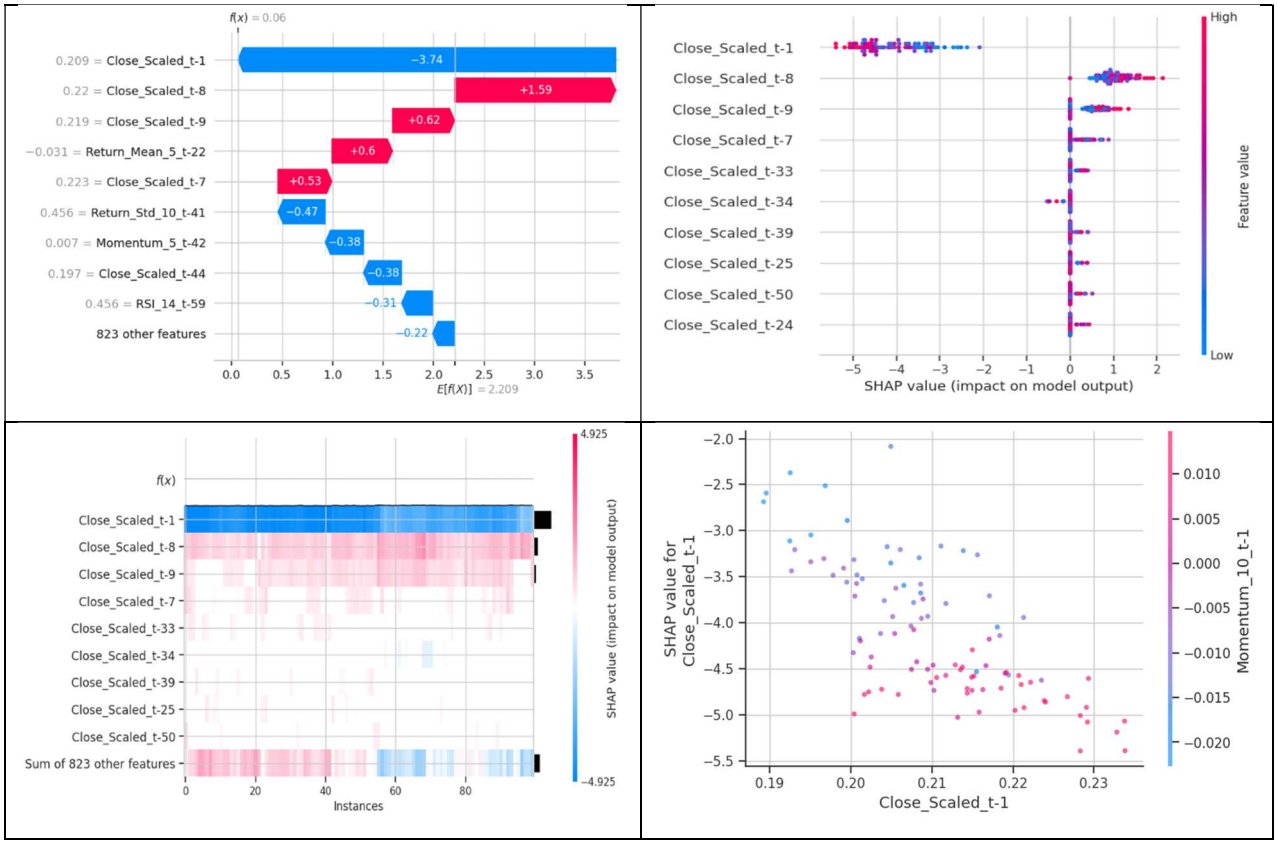


Figure 22: SHAP Waterfall Explanation of Individual Prediction Contributions., Figure 23: Distribution of SHAP Feature Importance Across Key Input Variables., Figure 24: Heatmap Visualization of Feature Impact Across Test Instances., & Figure 25: Dependence Plot of SHAP Values for the Scaled Closing Price Feature.

The SHAP heatmap shows that Close_Scaled_t-1, Close_Scaled_t-8, and Close_Scaled_t-9 remain the most active features over the explained instances (Figure 24). Finally, the SHAP dependence plot for Close_Scaled_t-1 shows a clear relationship between the recent scaled close value and the magnitude of its SHAP effect, with interaction colouring from Momentum_10_t-1 (Figure 25).

Overall, the explainability results support the model's forecasting behaviour: it primarily learns autoregressive price continuity from recent closing-price lags, while volatility, RSI, momentum, and moving-average features provide secondary adjustments.

The best-performing model, either iTransformer or TimeMachine, is automatically saved in the saved_models/ directory after comparative evaluation, while its architecture details, optimized hyperparameters, and metadata are stored in best_model_info.json. During initialization, the FastAPI backend dynamically restores the selected model and exposes /api/stocks and /api/predict/{stock} endpoints for real-time forecasting. The system uses stock symbols from locally stored CSV datasets to dynamically fetch live market data through yfinance while applying the same feature-engineering pipeline used during training. If live data retrieval is insufficient, the backend automatically falls back to local CSV data, ensuring reliable and uninterrupted forecasting.

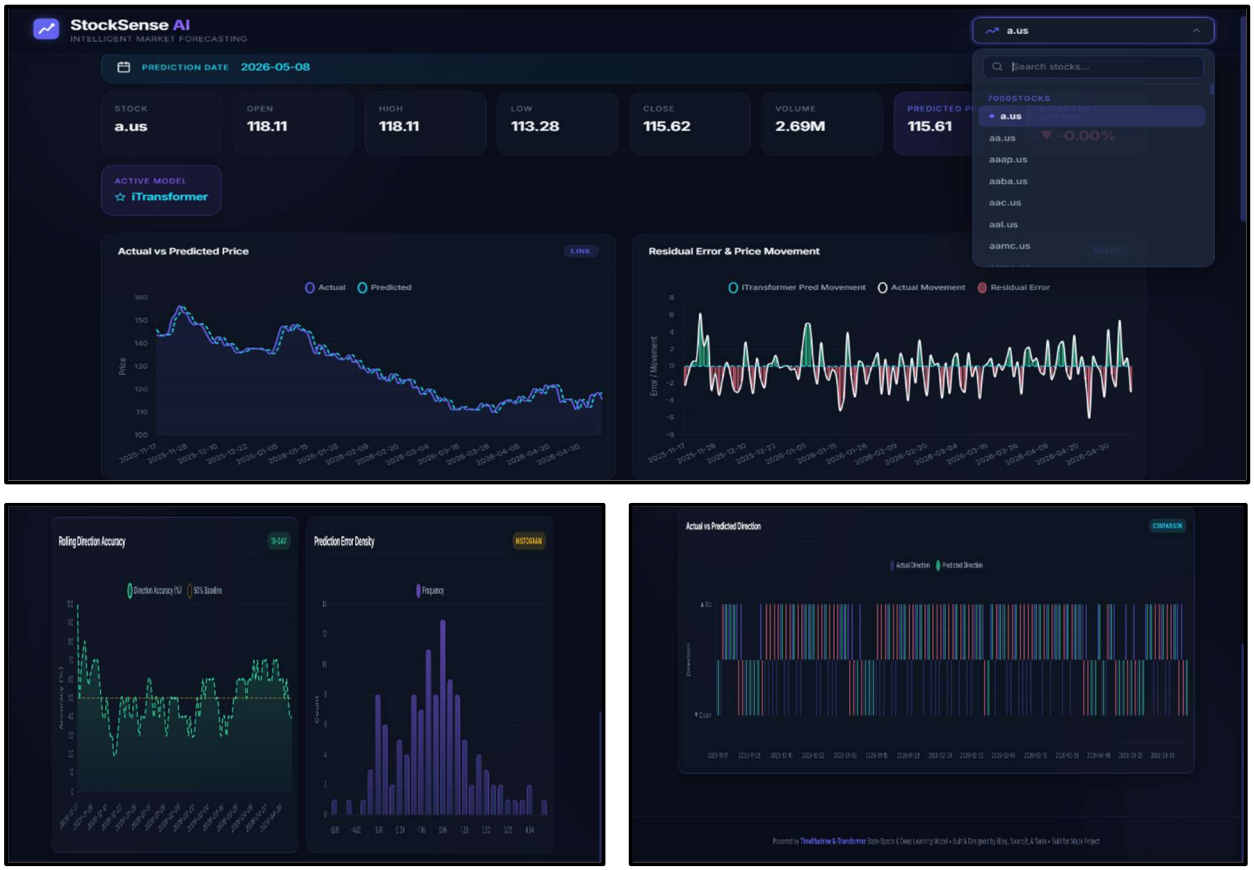


Figure 26: Interactive Stock Forecasting Dashboard with Real-Time Prediction Analytics., Figure 27: Rolling Directional Accuracy and Prediction Error Distribution Analysis., & Figure 28: Actual vs Predicted Market Direction Visualization in the Web Application.

The frontend dashboard integrates these backend predictions into an interactive stock-forecasting web application, displaying summary cards and five analytical visualizations including actual versus predicted price, residual error and price movement, rolling directional accuracy,

prediction error density, and actual versus predicted market direction. The complete deployment interface and visualization workflow are illustrated in Figure 26, Figure 27, and Figure 28.

Overall, the results show that deep sequence models can estimate normalized next-day stock prices with very low regression error, but directional movement remains difficult. iTransformer produced the best overall price-forecasting metrics, with RMSE of 0.019892, MAE of 0.011339, and R2 of 0.997801, and was therefore selected as the final model. TimeMachine remained competitive and slightly better for directional accuracy, but its overall composite score was lower.

The visual outputs and SHAP tables indicate that the model behaves conservatively and relies heavily on recent close-price lags. This is a strength for stable short-term price estimation, but it also explains why the model struggles to outperform random guessing on direction: when price movements are small and noisy, following recent close levels can minimize regression error while still missing the sign of the next movement. Therefore, the system is most suitable as a decision-support forecasting dashboard rather than a standalone trading signal generator.

7. CONCLUSION

This research presented a hybrid financial forecasting framework integrating iTransformer and TimeMachine (Mamba State-Space Model) architectures with Canonical Genetic Algorithm (CGA)-based hyperparameter optimization for multivariate stock-market prediction. The framework successfully combined long-term temporal dependency modeling, cross-variate attention learning, and evolutionary optimization within a unified deep-learning pipeline. A comprehensive preprocessing and feature-engineering strategy was implemented using multi-market datasets consisting of 211,560 records across 120 stocks from the 7000Stocks, NIFTY50, and US500 datasets.

Experimental evaluation demonstrated that both architectures achieved very low regression error in next-day stock-price forecasting. The iTransformer model achieved the best overall forecasting performance with RMSE = 0.019892, MAE = 0.011339, and $R^2 = 0.997801$, while TimeMachine showed slightly stronger directional prediction capability. The CGA optimization framework efficiently identified near-optimal hyperparameter configurations, improving model stability and reducing manual tuning effort. SHAP-based explainability analysis further confirmed that recent closing-price lags contributed most significantly to forecasting behavior, while volatility, RSI, and momentum indicators acted as secondary adjustment features.

The deployment of the selected model through a FastAPI backend and an interactive Chart.js-based dashboard demonstrated the practical applicability of the framework for real-time

financial forecasting. Overall, the proposed system provides an effective and scalable decision-support forecasting platform capable of handling heterogeneous financial datasets while maintaining high regression accuracy and deployment flexibility.

LIMITATIONS

Despite achieving strong regression performance, the framework exhibits several limitations inherent to financial time-series forecasting. First, directional prediction accuracy remained close to the random-walk threshold of 50%, indicating that accurately predicting short-term market direction under noisy and highly stochastic conditions remains challenging. The model primarily learned autoregressive price continuity from recent closing-price lags, which improved numerical forecasting accuracy but limited directional generalization.

Second, the forecasting pipeline relies heavily on historical OHLCV-based engineered features and does not incorporate external macroeconomic events, institutional order-flow data, geopolitical signals, or large-scale textual sentiment information. As a result, sudden regime shifts, black-swan events, and extreme market volatility may not be captured effectively.

Third, the Genetic Algorithm optimization process increases computational overhead because multiple candidate architectures must undergo repeated probe-training cycles during hyperparameter search. Additionally, although TimeMachine provides linear scalability compared with Transformer architectures, large-scale SHAP explainability analysis remains computationally expensive for long sequences and high-dimensional feature spaces.

Finally, the current deployment framework focuses primarily on short-horizon next-day forecasting and does not directly evaluate profitability, transaction costs, slippage, or portfolio-level trading performance in real-world market environments.

FUTURE SCOPE

The proposed framework can be extended in several directions to improve forecasting robustness, interpretability, and real-world applicability. Future work may integrate multimodal financial information such as news sentiment, economic calendars, earnings reports, social-media streams, and macroeconomic indicators using large language models and multimodal Transformer architectures. This could significantly improve directional forecasting capability during volatile market conditions.

Further research may also explore probabilistic forecasting frameworks capable of estimating predictive uncertainty alongside point forecasts. Integrating Bayesian deep learning or probabilistic state-space models could improve risk-sensitive financial decision making. Reinforcement learning and adaptive online-learning mechanisms may additionally enable continuous model updating under changing market regimes.

From an optimization perspective, advanced evolutionary approaches such as Differential Evolution, Particle Swarm Optimization, or Neuroevolutionary search strategies may provide more efficient hyperparameter exploration than traditional CGA methods. Ensemble learning approaches combining TimeMachine, iTransformer, and probabilistic architectures could also improve stability and directional consistency.

On the deployment side, the framework can be extended into a fully cloud-native distributed forecasting platform with GPU acceleration, automated retraining pipelines, streaming data ingestion, and broker API integration for paper trading or live algorithmic trading support. Future versions may also include portfolio optimization, risk management modules, and explainable AI dashboards for institutional financial analytics.

REFERENCES

[1] M. Al Ridhawi, M. Haj Ali, and H. Al Osman, "Stock Market Prediction Using Node Transformer Architecture Integrated with BERT Sentiment Analysis," *arXiv preprint arXiv:2603.05917*, 2026.

[2] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[3] J. Xiao, T. Deng, and S. Bi, "Comparative Analysis of LSTM, GRU, and Transformer Models for Stock Price Prediction," *arXiv preprint arXiv:2411.05790*, 2024

[4] X. Xu, Y. Liang, B. Huang, Z. Lan, and K. Shu, "Integrating Mamba and Transformer for Long-Short Range Time Series Forecasting," *arXiv preprint arXiv:2404.14757*, 2024.

[5] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long, "iTransformer: Inverted Transformers Are Effective for Time Series Forecasting," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.

[6] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[7] M. A. Ahamed and Q. Cheng, "TimeMachine: A Time Series is Worth 4 Mambas for Long-Term Forecasting," *arXiv preprint arXiv:2403.09898*, 2024.

[8] A. Mehdary, A. Chehri, A. Jakimi, and R. Saadane, "Hyperparameter Optimization with Genetic Algorithms and XGBoost: A Step Forward in Smart Grid Fraud Detection," *Sensors*, vol. 24, no. 4, p. 1230, Feb. 2024.

[9] J. Mun, "Comparative analysis of long short-term memory (LSTM) and transformer models for stock price prediction: A case study with Tesla data," *J. Eng. Sci. Technol.*, vol. 20, no. 6, 2025.

[10] A. Pessoa et al., "Mamba-ProbTSF: Probabilistic Time Series Forecasting with State Space Models," *PubMed Central (PMC)*, 2024.

[11] S. Shreyas, "A Case Study on Nifty 50 Using Genetic Algorithm-Based Hyperparameter Optimization for Stock Price Prediction," *ResearchGate*, 2026.

[12] Z. Zou, X.-X. Zhou, S.-J. Liu, and C.-Y. Hsu, "FT-iTransformer: A Stock Price Prediction Model Based on Time–Frequency Domain Collaborative Analysis," *ResearchGate*, 2026.

[13] Stanford University NLP Group, "Predicting Stock Market Trends from News Articles And Price Trends using Transformers," *Stanford University (CS224N Final Reports)*, 2024.

[14] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," *arXiv preprint arXiv:1803.01271*, 2018.

[15] A. Gu, K. Goel, and C. Ré, "Efficiently Modeling Long Sequences with Structured State Spaces," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2022.

[16] Y. Zhao and Z. Chen, “Forecasting stock price movement: new evidence from a novel hybrid deep learning model,” *JABES*, vol. 29, no. 2, pp. 91–104, Aug. 2021, doi: 10.1108/jabes-05-2021-0061.

[17] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A Time Series is Worth 64 Words: Long-term Forecasting with Transformers," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023.

[18] H. Zhou et al., "Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting," in *Proc. AAAI Conf. on Artificial Intelligence*, vol. 35, no. 12, pp. 11106-11115, 2021.

[19] SH. Wu, T. Hu, Y. Liu, H. Dong, J. Wang, and M. Long, "Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 22528-22541, 2021.

[20] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDformer: Frequency Enhanced Decomposed Transformer for Long-term Series Forecasting," in *Proc. Int. Conf. on Machine Learning (ICML)*, 2022.

[21] Y. Zhang and J. Yan, "Crossformer: Transformer Utilizing Cross-Dimension Dependency for Multivariate Time Series Forecasting," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023.

[22] A. Zeng, M. Chen, L. Zhang, and Q. Carmona, "Are Transformers Effective for Time Series Forecasting?," in *Proc. AAAI Conf. on Artificial Intelligence*, vol. 37, no. 9, pp. 11121-11128, 2023.

[23] N. Wu, B. Green, X. Ben, and S. O'Banion, "TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023.

[24] M. Jin et al., "Time-LLM: Time Series Forecasting by Reprogramming Large Language Models," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.

[25] B. N. Oreshkin, D. Carpov, N. Chapados, and Y. Bengio, "N-BEATS: Neural basis expansion analysis for interpretable time series forecasting," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2020.

[26] C. Challu, K. G. Olivares, B. N. Oreshkin, F. Garza, M. Mergenthaler-Canseco, and A. Dubrawski, "N-HiTS: Neural Hierarchical Interpolation for Time Series Forecasting," in *Proc. AAAI Conf. on Artificial Intelligence*, vol. 37, no. 6, pp. 6989-6997, 2023.

[27] Y. Yang, M. C. S. Uy, and A. Huang, "FinBERT: A Pretrained Language Model for Financial Communications," *arXiv preprint arXiv:2006.08097*, 2020.

[28] R. Akita, A. Yoshihara, T. Matsubara, and K. Uehara, "Deep learning for stock prediction using numerical and textual information," in *Proc. IEEE/ACIS 15th Int. Conf. on Computer and Information Science (ICIS)*, pp. 1-6, 2016.

PUBLICATIONS FROM THE WORK

The research work carried out in this project has contributed to an academic review paper titled "**AI Powered Stock Market Prediction Web Application - A Review**." The paper was accepted and presented at the International Conference on Frontiers of Engineering, Science and Technology, with Paper ID: 121, held on 12/12/2025. At the time of submission of this project report, the paper is under the publication process and yet to be formally published. The insights obtained from this research provided a strong theoretical foundation for the development of the proposed AI Powered Stock Market Prediction System.